

Universidad Argentina de la Empresa

Facultad de Ingeniería y Ciencias Exactas (FAIN)

Programación III

Trabajo Práctico Obligatorio

Sistema de organización de pedidos
y planificación logística

Propuesta 2

Primera entrega

Informe parcial de diseño e implementación

Integrantes

Ferreira, Martin - Legajo 1187544
Rodriguez, Lautaro - Legajo 1134799

Índice

1. Selección de Propuesta....3
2. Resolución de Ejercicios....3
2.1. Ejercicio 1.... 3
2.1.1. Descripción del Enunciado....3
2.1.2. Objetivo.... 3
2.1.3. Restricciones....3
2.1.4. Análisis del Modelo de Dominio (Los Datos)....4
2.1.5. Resolución del Ejercicio.... 4
2.2 Ejercicio 2 - Cálculo de la Ruta más Rápida....5
2.2.1. Descripción del Enunciado....5
2.2.2. Objetivo.... 5
2.2.3. Restricciones....5
2.2.4. Análisis del Modelo de Dominio (Los Datos)....5
2.2.5. Relaciones y Diseño....6
2.2.6. Resolución del Ejercicio.... 6
2.2.7. Test y Pruebas....7
2.2.8. Complejidad Temporal....8
2.3 Ejercicio 3.... 9
2.3.1. Descripción del Enunciado....9
2.3.2. Objetivo.... 9
2.3.3. Restricciones....9
2.3.4. Análisis del Modelo de Dominio (Los Datos)....9
2.3.5. Resolución del Ejercicio.... 10
3. Próxima Etapa.... 11

1. Selección de Propuesta

La propuesta elegida fue la segunda. Es una organización que distribuye alimentos, medicamentos e insumos entre centros comunitarios, todos unidos por conexiones que tienen un sentido de circulación y un tiempo de viaje.

La problemática que tiene es la gestión y optimización. Se debe optimizar lo máximo posible tanto la organización de los pedidos como el cálculo de las rutas que se van a llevar a cabo para mandar la ayuda lo mejor y de más calidad posible.

2. Resolución de Ejercicios

2.1. Ejercicio 1

2.1.1. Descripción del Enunciado

El ejercicio pide organizar y consultar los pedidos que recibe la organización. Cada pedido contiene un identificador, un centro de destino, una fecha límite, una prioridad y un peso total.

Se deben poder obtener los pedidos ordenados por fecha límite, comenzando por los más próximos, y por prioridad, comenzando por los más prioritarios. Cuando dos pedidos tienen la misma prioridad, se utiliza la fecha límite más cercana y, si también coinciden las fechas, el identificador menor. Además, se debe permitir buscar un pedido por su identificador.

2.1.2. Objetivo

El objetivo es facilitar la consulta de los pedidos y permitir encontrar uno en particular sin tener que recorrer siempre todos los elementos.

La entrada es un arreglo de pedidos y, para la búsqueda, un identificador. Las salidas son un nuevo arreglo ordenado según el criterio solicitado o el pedido correspondiente al identificador buscado. Si ese identificador no existe, la búsqueda devuelve null.

2.1.3. Restricciones

- Los identificadores de los pedidos deben ser únicos.
- El ordenamiento y la búsqueda deben implementarse sin utilizar los métodos de ordenamiento o búsqueda provistos por Java.
- El orden por fecha límite es ascendente.
- El orden por prioridad es descendente. Los empates se resuelven por fecha ascendente y luego por identificador ascendente.
- Deben contemplarse arreglos vacíos y búsquedas de identificadores inexistentes.

En la implementación también se valida que cada pedido tenga un centro de destino no vacío, una fecha límite y un peso positivo y finito.

2.1.4. Análisis del Modelo de Dominio (Los Datos)

El problema se modela mediante un arreglo de objetos Pedido. Cada objeto representa un pedido individual y reúne la información necesaria para ordenarlo, consultarlo y mostrarlo.

Las clases y sus responsabilidades son las siguientes:

- **Pedido:** representa los datos de un pedido: id, centroDestino, fechaLimite, prioridad y pesoTotal. La fecha se representa mediante LocalDate. Sus métodos permiten consultar los atributos y mostrar el pedido.
- **GestorPedidos:** conserva los pedidos ordenados por identificador. Permite consultar los pedidos por fecha o prioridad y buscar uno por su ID. También verifica que no existan identificadores repetidos.
- **OrdenamientoPedidos:** contiene la implementación de Merge-Sort y los criterios de comparación. Reúne las operaciones necesarias para ordenar los pedidos.
- **MainPrueba:** crea pedidos de ejemplo y muestra por consola los resultados de las consultas.
- **PruebasPedidos:** comprueba que el ordenamiento y la búsqueda produzcan los resultados esperados, incluyendo casos con arreglos vacíos y con un único pedido.

2.1.5. Resolución del Ejercicio

La estrategia seleccionada es divide y conquista, mediante Merge-Sort para ordenar y búsqueda binaria para consultar por identificador.

Merge Sort se encuentra en la clase OrdenamientoPedidos. El método ordenar copia el arreglo recibido e inicia el ordenamiento. El método mergeSort divide el segmento en dos mitades y ordena cada una recursivamente. Luego, el método merge combina ambas mitades en el orden establecido por el comparador.

La división termina cuando el segmento contiene un solo elemento o está vacío, porque en esos casos ya está ordenado. Durante la combinación, se comparan los elementos de ambas mitades y se incorpora primero el que corresponde según el criterio elegido.

Se seleccionó Merge-Sort porque mantiene un costo temporal de O(n log n) incluso en el peor caso. Quick-Sort también era una alternativa, pero puede alcanzar $O(n^2)$ cuando las particiones quedan muy desequilibradas.

Para buscar, el método buscarPorId de GestorPedidos utiliza una búsqueda binaria sobre pedidosPorId. Se compara el identificador buscado con el elemento central:

- Si coinciden, se devuelve ese pedido.
- Si el identificador buscado es menor, se continúa en la mitad izquierda.
- Si es mayor, se continúa en la mitad derecha.
- Si ya no quedan posiciones por revisar, se devuelve null.

En cada paso se descarta aproximadamente la mitad de los pedidos restantes. Por eso, la búsqueda tiene un costo temporal de O(log n) en el peor caso.

2.2 Ejercicio 2 - Cálculo de la Ruta más Rápida

2.2.1. Descripción del Enunciado

El siguiente ejercicio pide, dados una ubicación de origen y una de destino, obtener el recorrido de menor tiempo total entre ambas, mostrando las ubicaciones atravesadas y el tiempo que se tardó. Además, se deben poder deshabilitar las ubicaciones y volver a recalcular el tiempo y la ruta, informar cuando no exista una ruta disponible y poder visualizar todas las decisiones tomadas durante ese cálculo.

2.2.2. Objetivo

El objetivo, en síntesis, es minimizar la suma de los tiempos de las conexiones recorridas entre origen y destino. Recibe como entrada una red vial (grafo con ubicaciones y conexiones), un origen y un destino. La salida, si existe una ruta, es una lista ordenada de ubicaciones y tiempo total en minutos más otra lista de decisiones.

2.2.3. Restricciones

- Las conexiones son dirigidas.
- Los tiempos son enteros positivos.
- Solo se usan conexiones habilitadas.
- Puede no existir una ruta (un destino que está aislado, por ejemplo).

2.2.4. Análisis del Modelo de Dominio (Los Datos)

El problema se va a modelar como un grafo dirigido y ponderado, donde la red completa es el Grafo:

- Depósitos o Centros son los vértices.
- Las Conexiones viales con sentido son las aristas dirigidas.
- El tiempo de viaje es el peso de las aristas.

Las clases que componen a los datos y las responsabilidades son las siguientes:

- **TipoUbicación:** es un Enum que sirve únicamente para el tipo de ubicación que hay. En este sistema solo hay dos: Depósito o Centro Comunitario (DEPOSITO, CENTRO)
- **Ubicacion**: es una clase que da los datos de cada vértice: id, nombre y tipo, siendo el id el más importante de todos.
- **Conexion:** es la arista, tiene origen, destino, tiempo (peso) y si está o no habilitada.
- **RedVial:** guarda todas las ubicaciones en diccionarios y permite agregar, habilitar o deshabilitar una conexión. Es una clase importante ya que construye el Grafo que se le va a pasar a Dijkstra.
- **GrafoTDA:** es la interfaz con las operaciones de un grafo.
- **GrafoMatriz:** una clase que implementa al TDA para crear la matriz de adyacencia y un arreglo que relaciona cada posición de la matriz con el id del nodo. Cuando hay un 0 en la matriz significa que no hay una arista.
- **ResultadoRuta:** es el que orquesta la respuesta. Agrupa el origen, destino, si existe una ruta, el recorrido, el tiempo total y las decisiones de Dijkstra.

2.2.5. Relaciones y Diseño

La principal relación del proyecto a la de la clase RedVial la cual tiene Ubicacion y TipoUbicacion, tiene Conexion, para poder ver si está o no habilitada una conexión (arista), y por último está GrafoMatriz, la cual arma el grafo a partir de los datos que tiene RedVial para mandarselos a Dijkstra.

En lo que respecta a Decisiones la más importante es la Matriz de adyacencia, la cual permite consultar si hay o no una arista entre dos nodos y también poder saber su peso directamente. Esto se hace si son redes chicas, porque la complejidad es $O(n^2)$ y podría llegar a ser un problema si la red creciera mucho, sería una matriz enorme.

Por otro lado está la validación que tiene el modelo con los tiempos no positivos, los cuales son rechazados al momento de crear la conexión, entonces el algoritmo nunca podría recibir un grafo invalido. También, está el poder deshabilitar (y habilitar) una conexión en vez de borrarla. Cada vez que se calcula una ruta se reconstruye el grafo con las conexiones habilitadas, entonces si se deshabilita y se vuelve a recalcular es directo.

2.2.6. Resolución del Ejercicio

Sabiendo que el problema es de un camino mínimo desde origen en un grafo dirigido con pesos positivos, la mejor estrategia para esto es Greedy, más bien el algoritmo Dijkstra, ya que hay que encontrar, entre todos los caminos posibles del origen al destino, el que tiene la menor suma de pesos.

Dijkstra es un algoritmo Greedy: eso quiere decir que en cada paso elige, entre los vértices todavía no visitados o confirmados, el que tiene el menor tiempo conocido desde el origen y lo confirma como definitivo, sin volver a revisar la decisión.

La elección de Greedy se ve en la clase Dijkstra en el método seleccionarMinimo. Esto lo que hace es recorrer los nodos que no se visitaron, o sea vendrían a ser los pendientes, y devuelve el menor tiempo provisorio. Acá se puede ver el método:

```
private int seleccionarMinimo(GrafoTDA a, int v, Set<Integer> pendientes) {
  int minimo = -1;
  int pesoMinimo = Integer.MAX_VALUE;
  for (int n : pendientes) {
     if (a.existeArista(v, n) && a.pesoArista(v, n) < pesoMinimo) {
        pesoMinimo = a.pesoArista(v, n);
        minimo = n;
     }
  }
  return minimo;
}
```

La decisión que no tiene vuelta atrás es cuando el nodo que estaba en “pendientes” pasa a estar en “visitados”

Se elige Greddy porque al ser todos los tiempos positivos, si tenes un nodo con peso positivo, cualquiera hacia ese nodo tendría que pasar otro que pesa igual o más y sumas tramos positivos, así que nunca puede ser más corto. Esto lo garantiza Conexion en el modelo, ya que descarga tiempo menor o iguales a 0.

Otras alternativas que se sacaron fueron Floyd, ya que calculaba todos los pares, hacia $O(n^3)$ todo, y para un único origen era un exceso. También, se descartó usar Backtracking ya que ese sirve para listar todas las rutas, no para buscar la mínima solamente, era innecesario. Por esto mismo se termina eligiendo a Dijkstra.

2.2.7. Test y Pruebas

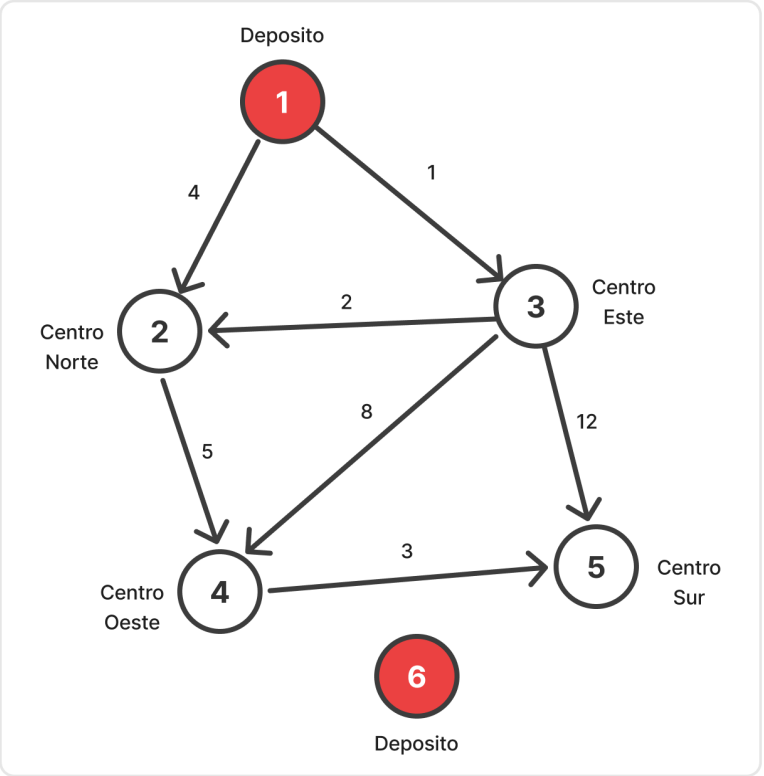
Todo comienza en la clase Test, la cual llama a Resolucion.calcularRutaMasRapida (red, origen, destino) e inicia el proceso.

Primero se valida que el origen y el destino existan, después si coinciden (origen == destino) para descartar ese caso particular de tiempo 0. Una vez que RedVial haya construido la red y GrafoMatriz hecho el grafo, se lo pasa a Dijkstra para resolver el problema del camino con el menor tiempo posible. Una vez terminado se reconstruye el recorrido que hizo y las decisiones para luego mostrarlas imprimiendolas.

Para hacer este Test, se creó una red de pruebas la cual demuestra visualmente un ejemplo de funcionamiento del algoritmo.

Paso	Confirma	2 Norte	3 Este	4 Oeste	5 Sur	6 Deposito
Inicio	1	4 (1)	1 (1)	∞	∞	∞
Vuelta 1	3	**3 (3)**	1 ✓	9 (3)	13 (3)	∞
Vuelta 2	2	3 ✓	1 ✓	**8 (2)**	13 (3)	∞
Vuelta 3	4	3 ✓	1 ✓	8 ✓	**11 (4)**	∞
Vuelta 4	5	3 ✓	1 ✓	8 ✓	11 ✓	∞
Vuelta 5	— (`-1` , corta)					∞

Red usada en la pruebas



2.2.8. Complejidad Temporal

Siendo n la cantidad de vértices y m la cantidad de aristas, la complejidad podría ser $O(n^2 + m \cdot n)$, pero dentro del método “ejecutar” en Dijkstra cada búsqueda es lineal, o sea (O(n), lo que termina dando un $O(n^3)$. El total entonces de una consulta es $O(n^3)$.

2.3 Ejercicio 3

2.3.1. Descripción del Enunciado

El ejercicio pide seleccionar los lotes que se cargarán en un vehículo con capacidad limitada. Cada lote disponible tiene un identificador, un peso entero positivo y un puntaje de prioridad entero no negativo.

La selección debe obtener la mayor prioridad total posible sin superar la capacidad del vehículo. Los lotes no pueden fraccionarse y cada uno puede cargarse una sola vez. Se deben mostrar los lotes seleccionados, el peso total, la prioridad obtenida y la capacidad restante.

2.3.2. Objetivo

El objetivo es maximizar la suma de las prioridades de los lotes seleccionados, respetando la capacidad máxima del vehículo.

La entrada es un arreglo de lotes disponibles y una capacidad máxima. La salida es una selección de lotes junto con su peso total, prioridad total y capacidad restante.

No es obligatorio llenar completamente el vehículo. Una selección puede dejar capacidad libre y aun así ser óptima si ninguna otra combinación válida obtiene una prioridad mayor.

2.3.3. Restricciones

- El peso de cada lote debe ser un número entero positivo.
- La prioridad de cada lote debe ser un número entero no negativo.
- Los lotes son indivisibles: se cargan completos o no se cargan.
- Cada lote puede seleccionarse como máximo una vez.
- El peso total seleccionado no puede superar la capacidad del vehículo.
- La capacidad se representa mediante un entero no negativo, se contempla el caso de capacidad cero.
- Cada posición del arreglo debe representar un lote distinto.
- La selección de cargas se resuelve por separado de la planificación del recorrido.

2.3.4. Análisis del Modelo de Dominio (Los Datos)

El problema se modela como una mochila. La capacidad del vehículo equivale a la capacidad de la mochila, mientras que cada lote equivale a un objeto que tiene un peso y un valor. En este caso, el valor es su prioridad.

La denominación 0/1 indica que cada lote tiene dos posibilidades: no seleccionarse o seleccionarse una vez.

Las clases y sus responsabilidades son las siguientes:

- **Lote:** representa un lote disponible mediante los atributos id, peso y prioridad. Valida que el peso sea positivo y que la prioridad no sea negativa.
- **SeleccionCargas:** contiene el algoritmo de programación dinámica. Su método seleccionar calcula la prioridad máxima y su método reconstruir identifica los lotes que permiten obtenerla.
- **ResultadoCarga:** agrupa los lotes seleccionados, el peso total, la prioridad total y la capacidad restante.
- **MainPrueba:** prepara un ejemplo y muestra la selección obtenida.
- **PruebasCargas:** verifica casos como una selección óptima conocida, capacidad cero, ausencia de lotes, lotes que no entran y empates de prioridad.

La capacidad del vehículo se recibe como un número entero. Para esta funcionalidad no se creó una clase Vehiculo.

2.3.5. Resolución del Ejercicio.

La estrategia seleccionada es programación dinámica, aplicada al problema de la mochila.

Esta estrategia permite resolver el problema a partir de soluciones más pequeñas. En lugar de calcular repetidamente todas las combinaciones, se guarda el mejor resultado para cada grupo de lotes y cada capacidad disponible.

El algoritmo recorre la matriz por filas. Para cada lote y capacidad se consideran los siguientes casos:

- **Primer lote:** si entra, se guarda su prioridad; si no entra, se guarda cero.
- **El lote actual no entra:** se conserva el resultado de la fila anterior para esa capacidad.
- **El lote actual entra:** se comparan dos alternativas: no cargarlo, conservando el resultado anterior, o cargarlo, sumando su prioridad al mejor resultado para la capacidad restante.

Cuando se evalúa cargar un lote, se consulta la fila anterior. Esto garantiza que ese mismo lote no pueda utilizarse nuevamente dentro de la combinación.

Al completar la tabla, la última fila en la columna de capacidad máxima contiene la mayor prioridad posible. Sin embargo, todavía es necesario identificar los lotes seleccionados.

Para hacerlo, el método “reconstruir” recorre la matriz desde la última fila hacia la primera. Si el valor actual es distinto del valor de la fila anterior, significa que incorporar ese lote mejoró la solución. Entonces se lo selecciona y se descuenta su peso de la capacidad pendiente.

Por ejemplo, con capacidad 8 y lotes de pesos 1, 4, 5 y 7, cuyas prioridades son 10, 3, 5 y 8, la solución selecciona los lotes de peso 1 y 7. El resultado tiene peso total 8, prioridad total 18 y capacidad restante 0.

Se eligió programación dinámica porque una decisión greedy no garantiza obtener la mejor combinación. Por ejemplo, con capacidad 10, seleccionar primero un lote de peso 6 y prioridad 9 impide cargar dos lotes de peso 5 y prioridad 8 cada uno,

que juntos alcanzarían una prioridad de 16.

3. Próxima Etapa

Para la siguiente etapa se requiere seleccionar tres funcionalidades adicionales dada un conjunto de 7 de ellas, una del bloque A y dos del Bloque B. Las que seleccionamos fueron las siguientes:

- A1: Consultas de rutas entre todos los centros.
- B1: Recorrido con múltiples entregas.
- B2: Distribución de cargas entre vehículos.